

Set 3 - Endsem Lab Exam (SEC-A)

Instructions:

1. Attempt **any one** of the questions given below.
2. Use only the following editors: nano, vim, gedit, or nedit.
3. File naming in this format - **SetNo_QNo_RollNo.c**, e.g. 3_1_2025123.c
4. **Google Form Link For Submission:** <https://forms.gle/sV8VeqWEg5tfphAp9>

Question 1: Min Heap Construction and Deletion [CO1,CO2,CO3,CO4]

Implement a **Min Heap** using an array. You are given a fixed `main()` function that takes input for n elements and inserts them into the heap. Your task is to complete the following functions:

- `insert()` — to insert an element while maintaining Min Heap property
- `heapify()` — to restore heap property after deletion
- `deleteMin()` — to remove and return the minimum element

Sample Input:

```
10
35 33 42 10 14 19 27 44 26 31
```

Sample Output:

```
Heap elements before deletion:
10 14 19 26 31 42 27 44 33 35
Minimum element: 10
Heap elements after deletion:
14 26 19 33 31 42 27 44 35
```

C Base Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Heap {
4     int* array;
5     int capacity;
6     int size;
7 };
8 struct Heap* createHeap(int capacity) {
9     struct Heap* heap = (struct Heap*)malloc(sizeof(struct Heap));
10    heap->array = (int*)malloc(capacity * sizeof(int));
11    heap->capacity = capacity;
12    heap->size = 0;
13    return heap;
14 }
15 void swap(int* a, int* b) {
16     int temp = *a;
17     *a = *b;
18     *b = temp;
19 }
20 /* Complete this function */
21 void heapify(struct Heap* heap, int i) {
22     // Your code here
23 }
24 /* Complete this function */
25 void insert(struct Heap* heap, int value) {
26     // Your code here
27 }
28 void printHeap(struct Heap* heap) {
29     for(int i = 0; i < heap->size; i++)
30         printf("%d ", heap->array[i]);
31     printf("\n");
32 }
33 /* Complete this function */
34 int deleteMin(struct Heap* heap) {
35     // Your code here
36 }
37 int main() {
38     int n, x;
39     scanf("%d", &n);
40     struct Heap* heap = createHeap(n);
41     for(int i = 0; i < n; i++) {
42         scanf("%d", &x);
43         insert(heap, x);
44     }
45     printf("Heap elements before deletion:\n");
46     printHeap(heap);
47     int min = deleteMin(heap);
48     printf("Minimum element: %d\n", min);
49     printf("Heap elements after deletion:\n");
50     printHeap(heap);
51     return 0;
52 }
```

Solution Code

```
1 void heapify(struct Heap* heap, int i) {
2     int smallest = i;
3     int left = 2*i + 1;
4     int right = 2*i + 2;
5
6     if (left < heap->size && heap->array[left] < heap->array[smallest])
7         smallest = left;
8
9     if (right < heap->size && heap->array[right] < heap->array[smallest])
10        smallest = right;
11
12    if (smallest != i) {
13        swap(&heap->array[i], &heap->array[smallest]);
14        heapify(heap, smallest);
15    }
16 }
17
18 void insert(struct Heap* heap, int value) {
19     int i = heap->size++;
20     heap->array[i] = value;
21
22     while (i != 0 && heap->array[(i-1)/2] > heap->array[i]) {
23         swap(&heap->array[i], &heap->array[(i-1)/2]);
24         i = (i-1)/2;
25     }
26 }
27
28 int deleteMin(struct Heap* heap) {
29     if (heap->size == 0) {
30         printf("Heap is empty.\n");
31         return -1;
32     }
33
34     int min = heap->array[0];
35
36     heap->array[0] = heap->array[heap->size - 1];
37     heap->size--;
38
39     heapify(heap, 0);
40
41     return min;
42 }
```

Question 2: Hashing with Linear Probing and Double Hashing [CO1,CO2,CO3,CO4]

Implement hash tables using **open addressing**. You are required to insert a given set of keys using the following collision-resolution methods.

Part A: Linear Probing Use a hash table of size $m = 16$ with the hash function:

$$h(k, i) = (h'(k) + i) \bmod m$$

where the primary hash function uses the multiplication method:

$$h'(k) = ((A \cdot k) \bmod 2^w) \gg (w - r)$$

where $m = 16 = 2^r$, so $r = 4$, $w = 32$, $\gg$ denotes the bitwise right-shift operator and A is a odd integer, $2^{w-1} < A < 2^w$ choose $A = 2654435769$ (fixed odd constant)

Part B: Double Hashing

Use a separate hash table of size $m = 11$ (prime number) with the probe function:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

where:

$$h_1(k) = k \bmod m$$

$$h_2(k) = 1 + (k \bmod (m - 1))$$

Note: Since $m = 11$ is prime, the secondary hash value is always non-zero and produces a valid probing sequence. You must insert all given keys using Linear Probing in the first table, insert all given keys using Double Hashing in the second table and Display the final hash table for each method

Sample Input:

```

9
10 22 31 4 15 28 17 88 59
Sample Output:
Hash Table using Linear Probing:
-1 -1 10 31 15 28 88 4 17 22 59 -1 -1 -1 -1
Hash Table using Double Hashing:
22 -1 59 17 4 15 28 88 -1 31 10

```

C Base Code

```

1 #include <stdio.h>
2 #define SIZE1 16
3 #define SIZE2 11
4 void initTable(int table[], int size) {
5     for(int i = 0; i < size; i++)
6         table[i] = -1;
7 }
8 /* Complete this function */
9 void linearProbingInsert(int table[], int key) {
10     // Your code here
11 }
12 /* Complete this function */
13 void doubleHashingInsert(int table[], int key) {
14     // Your code here
15 }
16 void printTable(int table[], int size) {
17     for(int i = 0; i < size; i++)
18         printf("%d ", table[i]);
19     printf("\n");
20 }
21 int main() {
22     int n, keys[100];
23     scanf("%d", &n);
24     for(int i = 0; i < n; i++)
25         scanf("%d", &keys[i]);
26     int table1[SIZE1], table2[SIZE2];
27     initTable(table1, SIZE1);
28     initTable(table2, SIZE2);
29     for(int i = 0; i < n; i++) {
30         linearProbingInsert(table1, keys[i]);
31         doubleHashingInsert(table2, keys[i]);
32     }
33     printf("Hash Table using Linear Probing:\n");
34     printTable(table1, SIZE1);
35     printf("\nHash Table using Double Hashing:\n");
36     printTable(table2, SIZE2);
37     return 0;
38 }

```

Solution Code

```

1 void linearProbingInsert(int table[], int key) {
2     int i = 0;
3     int index;
4
5     unsigned int A = 2654435769U;
6     int w = 32;
7     int r = 4;
8
9     int h = ((unsigned int)(A * key)) >> (w - r);
10
11     while(i < SIZE1) {
12         index = (h + i) % SIZE1;
13
14         if(table[index] == -1) {
15             table[index] = key;
16             return;
17         }
18         i++;
19     }
20 }
21
22 void doubleHashingInsert(int table[], int key) {
23     int i = 0;
24     int index;
25
26     int h1 = key % SIZE2;
27     int h2 = 1 + (key % (SIZE2 - 1));
28
29     while(i < SIZE2) {
30         index = (h1 + i * h2) % SIZE2;
31
32         if(table[index] == -1) {
33             table[index] = key;
34             return;
35         }
36         i++;
37     }
38 }

```

Marking Scheme

Question 1: Min Heap Construction and Deletion (10 Marks)

1. Correct implementation of `insert()` maintaining Min Heap property – 3 Marks
2. Correct implementation of `heapify()` (recursive adjustment of subtree) – 2 Marks
3. Correct implementation of `deleteMin()` (root replacement and size update) – 2 Marks
4. Proper handling of empty heap condition in `deleteMin()` – 1 Mark
5. Correct use of `heapify()` after deletion – 1 Mark
6. Correct output for given input test case – 1 Mark

Question 2: Hashing with Linear Probing and Double Hashing (10 Marks)

1. Correct implementation of Linear Probing insertion logic using multiplication-method hash $h(k, i) = (h'(k) + i) \bmod 16$ – 3 Marks
2. Correct implementation of Double Hashing insertion logic using $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 11$ – 3 Marks
3. Correct computation of multiplication-method primary hash function $h'(k) = ((A \cdot k) \bmod 2^{32}) \gg (32 - 4)$ – 1 Mark
4. Correct computation of double hashing functions $h_1(k) = k \bmod 11$ and $h_2(k) = 1 + (k \bmod 10)$ – 1 Mark
5. Proper handling of collisions using probing sequence and insertion in empty slot – 1 Mark
6. Correct final output for the given input test case – 1 Mark